ASSIGNMENT - 13 .2

Thumma Hasini | 2303A52076 | Batch - 37

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions.

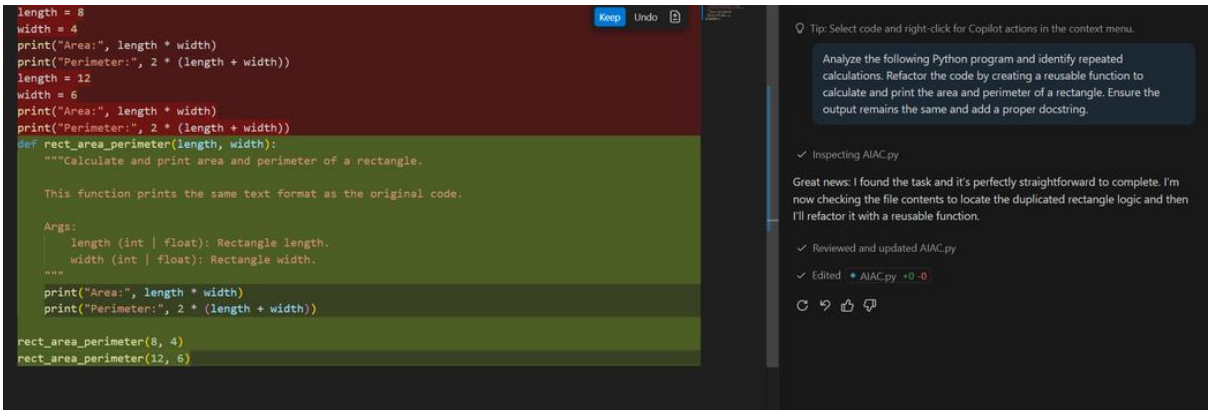
Sample Code:

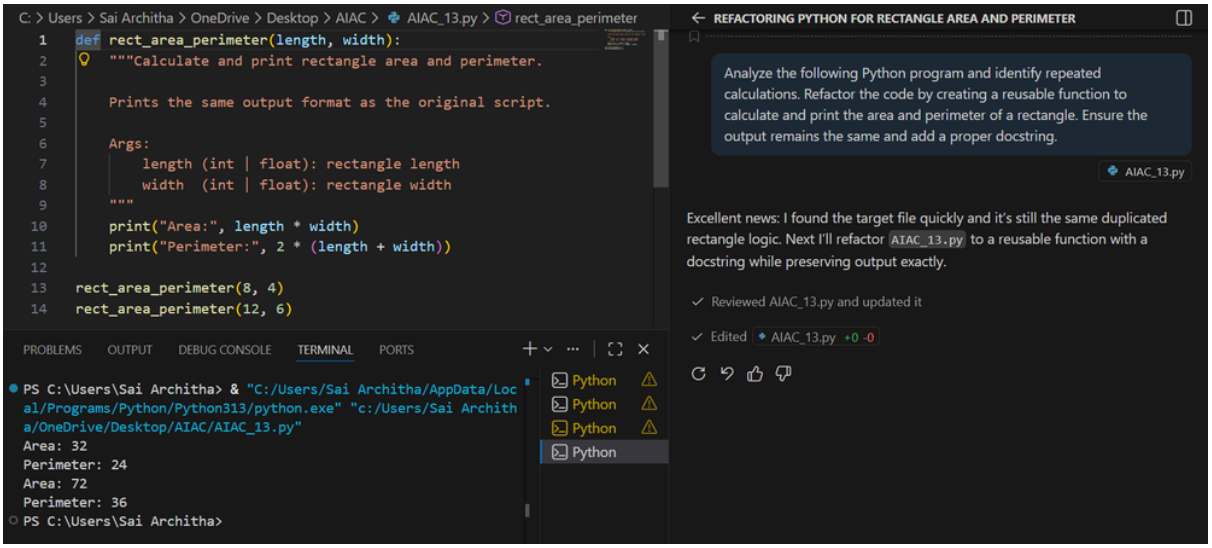
```
length = 8

width = 4

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))

length = 12

width = 6

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

Prompt Used: Analyze the following Python program and identify repeated calculations. Refactor the code by creating a reusable function to calculate and print the area and perimeter of a rectangle. Ensure the output remains the same and add a proper docstring.





Explanation: The repeated calculations for area and perimeter were placed inside a reusable function called rectangle_details(). This improves code readability, reduces repetition, and makes the program easier to maintain.

Task Description -2 (Refactoring – Modularizing Inline calculations)

Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

• Identify repeated calculation patterns in the code.

• Convert them into reusable functions.

• Ensure readability and proper documentation.

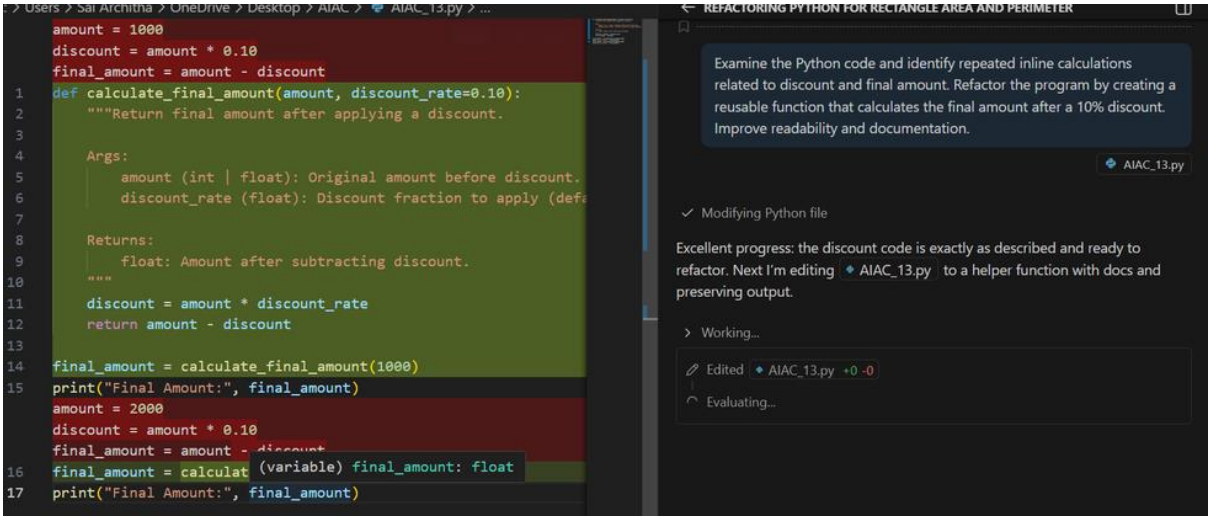
Sample Code:

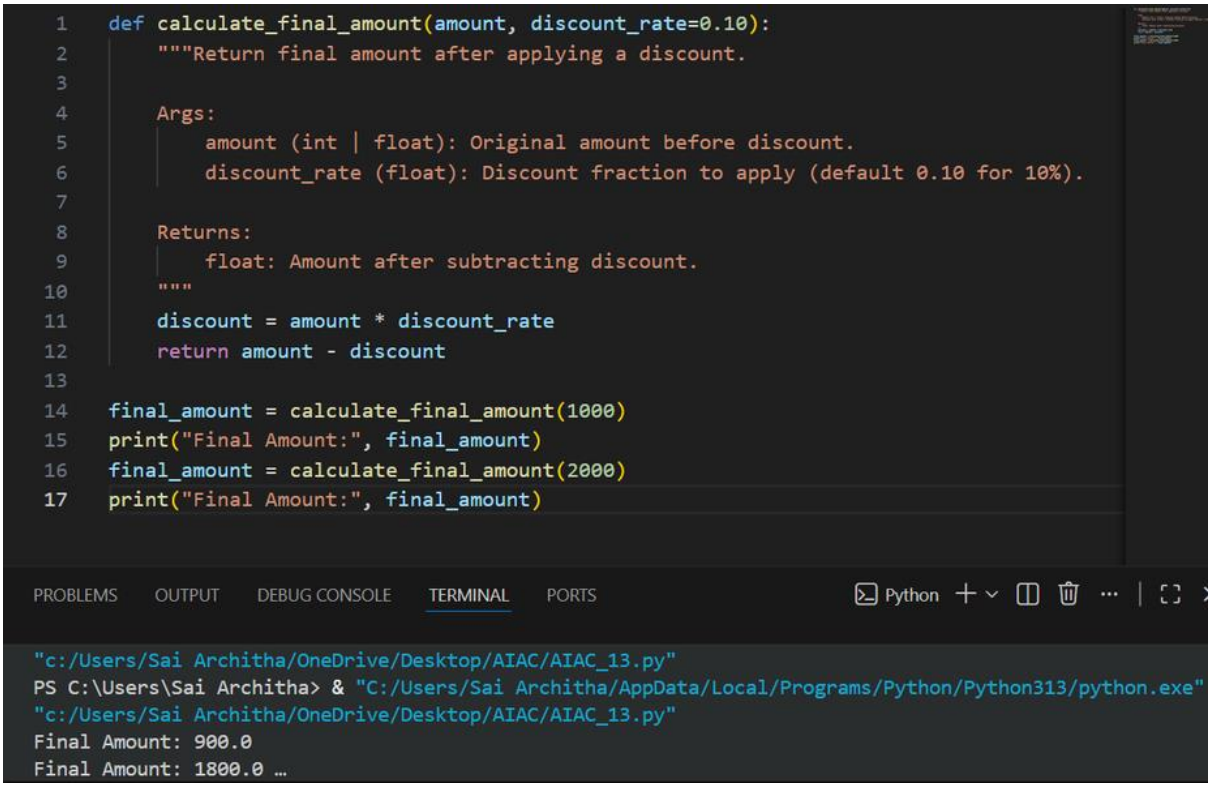
```
amount = 1000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)
```

Expected Output:

A modular Python program that uses a reusable function to compute final amounts for different inputs.

Prompt Used: Examine the Python code and identify repeated inline calculations related to discount and final amount. Refactor the program by creating a reusable function that calculates the final amount after a 10% discount. Improve readability and documentation.





Explanation: The repeated discount calculation logic was moved into a reusable function calculate_final_amount(). This makes the program modular and avoids repeating the same code.

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

• Use AI to recommend more efficient data structures or logic simplifications.

• Replace inefficient constructs while preserving program behavior.

• Compare execution efficiency before and after refactoring.

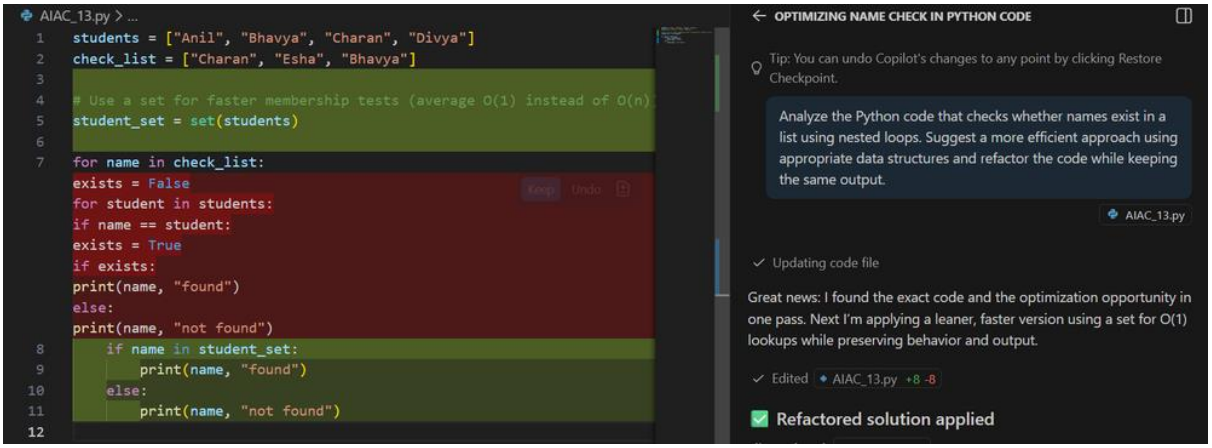
Sample Code:

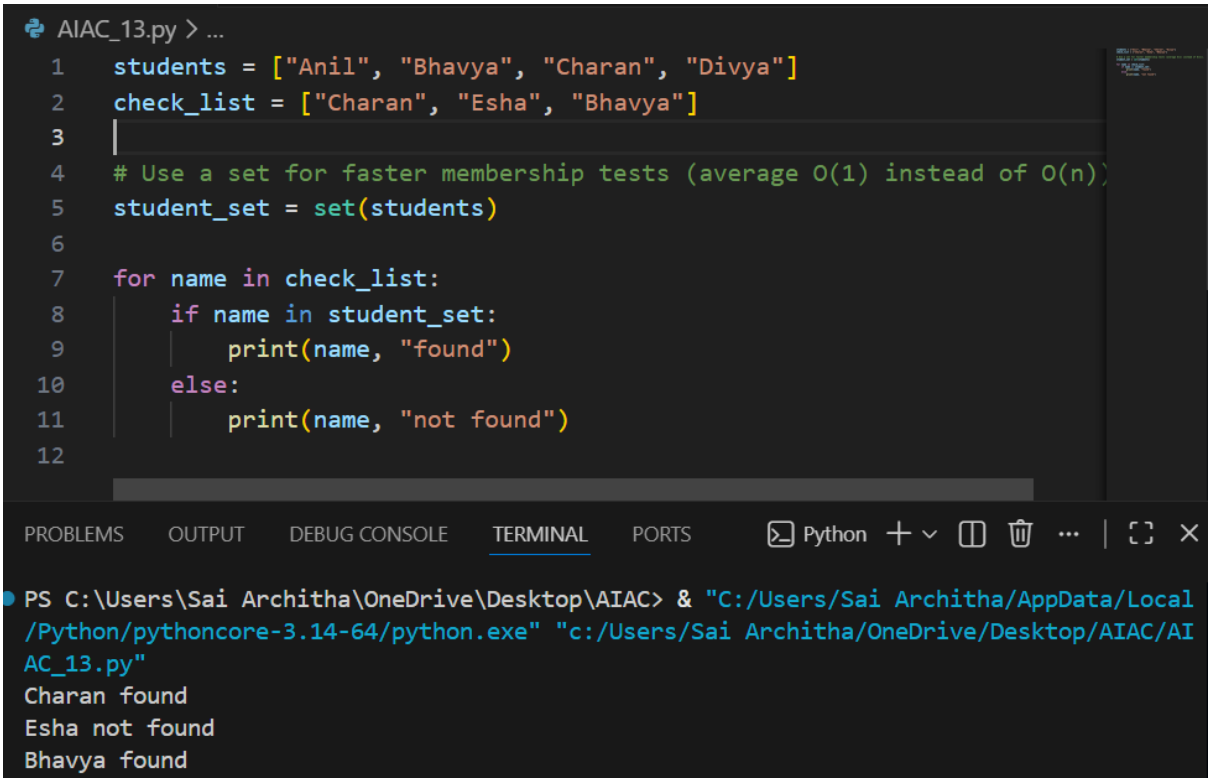
```
students = ["Anil", "Bhavya", "Charan", "Divya"]

check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:

exists = False

for student in students:

if name == student:

exists = True

if exists:

print(name, "found")

else:

print(name, "not found")
```

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

Prompt Used: Analyze the Python code that checks whether names exist in a list using nested loops. Suggest a more efficient approach using appropriate data structures and refactor the code while keeping the same output.





Explanation: The nested loop was inefficient because it compared each name multiple times. By converting the students list into a set, the lookup becomes faster. This improves performance and simplifies the logic.

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing results.

Sample Code:

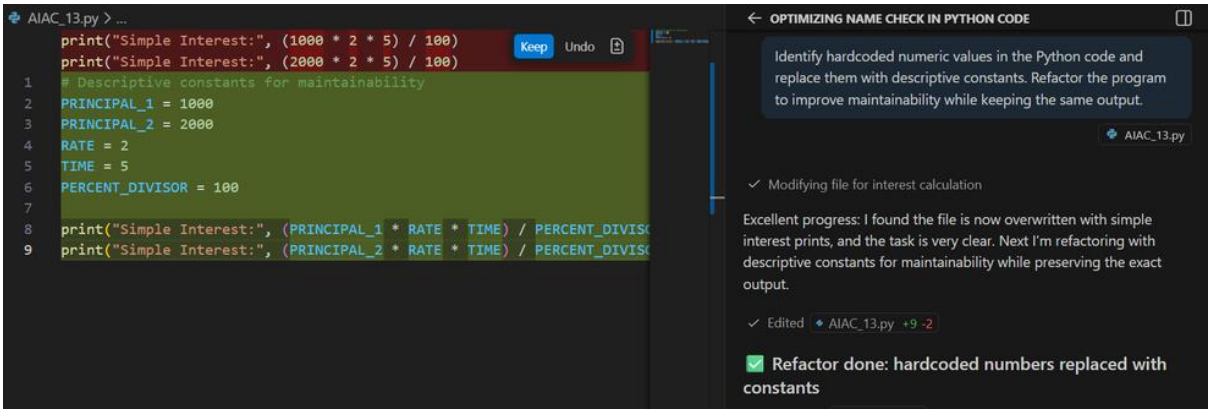
print("Simple Interest:", (1000 * 2 * 5) / 100)

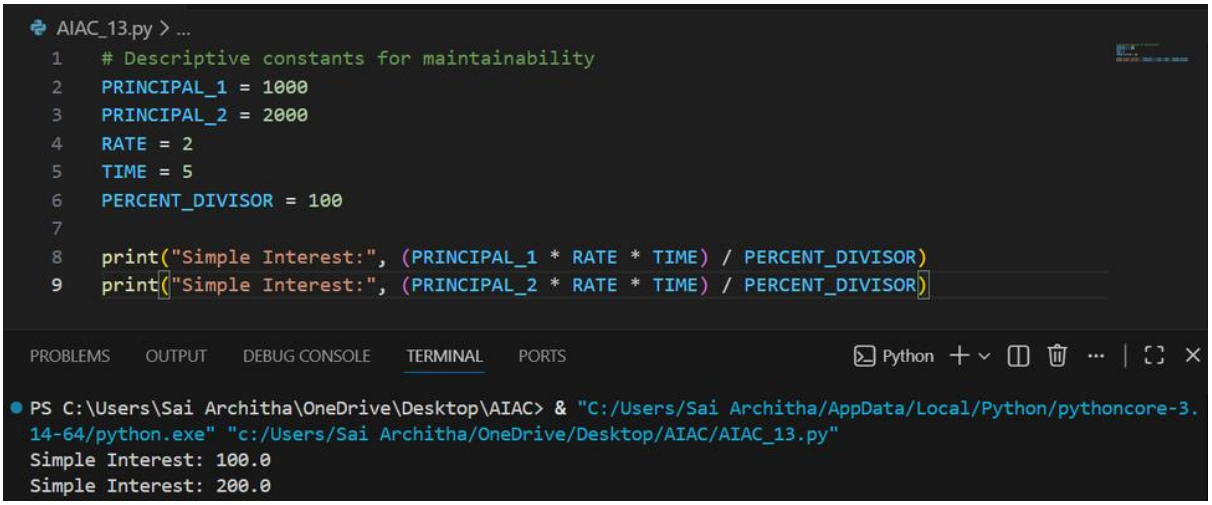
print("Simple Interest:", (2000 * 2 * 5) / 100)

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations.

Prompt Used: Identify hardcoded numeric values in the Python code and replace them with descriptive constants. Refactor the program to improve maintainability while keeping the same output.





Explanation: Hardcoded values such as interest rate and time were replaced with constants (RATE, TIME). This makes the program easier to update and improves readability.

Task Description -5 (Refactoring – Enhancing Variable Naming and Code Clarity)

Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments.

Instructions:

• Replace non-descriptive variable names with meaningful identifiers.

• Add inline comments where the logic may be unclear.

• Ensure program output remains the same.

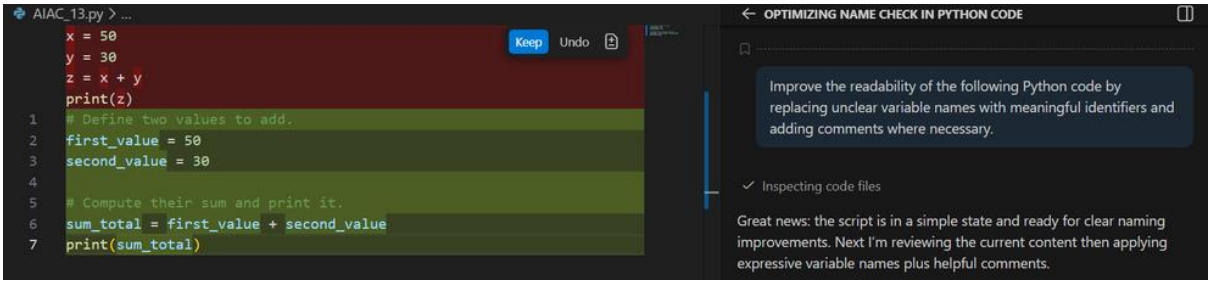
Sample Code:

```
x = 50

y = 30

z = x + y

print(z)
```

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

Prompt Used: Improve the readability of the following Python code by replacing unclear variable names with meaningful identifiers and adding comments where necessary.



Explanation: The variables x, y, and z were replaced with meaningful names (num1, num2, total_sum). Comments were added to explain each step, improving readability.